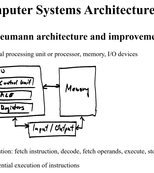
Computer Systems Architectures

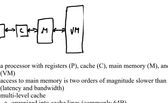
von Neumann architecture and improvements

- central processing unit or processor, memory, I/O devices



- operation: fetch instruction, decode, fetch operands, execute, store
- sequential execution of instructions
- memory bottleneck
- improvements:
  - memory hierarchy
  - hardware parallelism

Memory hierarchy



- a processor with registers (P), cache (C), main memory (M), and virtual memory (VM)
- access to main memory is two orders of magnitude slower than access to registers (latency and bandwidth)
- multi-level cache
  - organized into cache lines (commonly 64B)
  - entire cache block is transferred
  - if data is missing, the processor still waits
  - exploiting spatial and temporal locality
  - cache hits and misses
    - example: accessing a matrix
  - where in cache to copy data from main memory
    - delete the oldest cache line
    - associative: all locations are equivalent
    - set-associative: a subset of available locations
    - direct: exact mapping
  - ensuring memory coherence is important
    - write-through: when writing to cache, data is also written to main memory
    - write-back: dirty bit is set in cache line when writing, when cache line is about to be overwritten, it's content is first written to the main memory
  - multi-level cache system (L1, L2, L3)
- virtual memory
  - extension of main memory
  - each process sees its own private address space
  - memory is divided into pages for simpler OS management (typical size 2MB)
  - page swapping and page faults when the size of virtual memory is grater than the size of physical
    - significantly slower access than to main memory (1,000,000 times), even greater need for data locality
  - rapid virtual to physical memory address mapping with the help of translation lookaside buffer (TLB)
    - specialized cache for virtual to physical address translation (up to 128 entries on 1st level)
    - TLB hit
      - no need to access page table
    - TLB miss
      - access to a page not in TLB, element by element walk over the whole page table to find translation
      - high TLB miss rate when accessing memory elements even when number of page faults is low (but higher than the size of TLB)

Hardware parallelism

- pipelining
  - functional units are arranged into stages
  - stages should be as equally complex as possible
  - example: summation of two floating point numbers
    - $1.23e4 + 6.54e3$
    - loading, exponent comparison, exponent alignment, summation, normalization, rounding, storing
    - number of cycles for 1 and 1000 summations
- vector instructions
  - explicit parallel usage of multiple functional units on a small amount of data
  - special instructions
- parallel functional units
  - exploit parallelism within a single instruction stream
  - example: two units, one is working on odd and one on even iterations of a for loop
- very large instruction word (VLIW)
  - at the compile time many independent operations are grouped together
  - instruction words 64 – 1024 bits
- superscalar execution
  - parallel execution of independent instructions
  - allocation of functional units is dynamic, at runtime (for non-superscalar processors is static, at compile time)
  - examples:
    - parallel arithmetic units with dynamic allocation
    - speculative execution
      - during conditional execution, instructions that follow can already be executed

        ```
        if (x > 0) {
          z = x + y;
          w = x;
        }
        else {
          w = y;
        }
        ```

      - if `(x > 0)` and assignment `z = x + y` are performed in parallel
- hardware multithreading
  - no need to search for parallelism in sequential thread (in many cases does not exist)
  - simultaneous multithreading (instructions from multiple threads are fed to superscalar scheduler dynamically, at runtime
  - switch-on event multithreading
    - latency hiding
    - when one thread stops due to memory or IO access, another can be processed
    - rapid switching of threads at the long-latency operations

Serial illusion

- developments in hardware have led to long-sustained serial illusion
- serial traps: serial assumptions incorporated in tools and thinking

```
void addme(int n, double a[n], double b[n], double c[n]) {
  for (int i = 0; i < n; i++)
    a[i] = b[i] + c[i];
}

double a[10]
addme(9, a+1, a, a);  // pointer arithmetic causing aliasing
```

- compilers are not reliable at discovering parallelism (loops, memory access via pointers in C)

Flynn's characterization

- division with respect to instruction and data flow:
  - SISD: standard non-parallel (scalar) processor
  - SIMD: array processor (array of functional units and a shared controller)
  - MIMD: separate instructions streams operating on separate data
    - multi-core, multi-processor, multiple computers, heterogeneous computer
    - shared and distributed memory and combinations
  - MISD: only theoretical (systolic arrays)
  - SIMT: single instruction multiple threads (inside warp)
    - tiled SIMD: each processor emulates multiple threads using masking, blocks of threads share a control processor, divergent flow is not beneficial
    - preferred to access the same cache line for performance improvement (opposite to classical multi-core systems)

Vector Processor

- single instruction, multiple data
- single-core processor (P) extended with many processing elements (PE) working in parallel on vector data

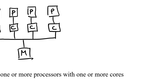
- first supercomputers, long and expensive development
- data parallel problems, operate on vectors
- scalar support
- vector support
  - registers, functional units, instructions
  - operand alignment in memory
  - memory interleaving for faster access
  - size of problem must fit to the hardware
  - branching and masking
    - slow execution
    - packing groups of threads with the same result of condition to avoid masking

      ```
      if (a & 1)
        a = 3*a + 1;
      else
        a = a/2;
      ```

- poor scalability

Shared-memory systems

- multiple instructions, multiple data



- have one or more processors with one or more cores
- processors are connected to shared memory via a bus
- processors share the same memory
- changes in memory are visible to all processors
- memory is divided into modules; only one processor can access a module at a time
- each processor has its own L1 cache, because it stores both data and instructions
- L2 and L3 caches store only data, so they can be shared by multiple processors
- uniform address space
- UMA and NUMA architectures
  - complexity and scalability
  - memory access times
  - ensuring memory coherence

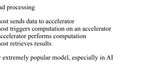
Distributed-memory systems

- multiple instructions, multiple data

- processors have their own memory
- other processors do not see memory changes
- processors exchange data by sending messages
- slower compared to the shared-memory systems
- more scalable
- focus on interconnections
- cost effective
- off-the-shelf hardware components
- types
  - cluster
    - commodity network components (Ethernet, Infiniband)
    - each node runs its own OS
  - massively parallel processors (MPPs)
    - more tightly integrated
    - can run the same instance of OS on all nodes
    - treats the whole machine as single OS from admin point of view
    - more commonly only special interconnects
  - constellations
    - more cores per node than nodes
    - spatial node sharing among users

Accelerators (GPUs)

- single instruction, multiple threads



- offload processing
  - host sends data to accelerator
  - host triggers computation on an accelerator
  - accelerator performs computation
  - host retrieves results
- today extremely popular model, especially in AI
- hierarchical design

Architecture of a modern computer system

- heterogenous systems
  - shared-memory within a node
  - accelerators on some nodes
  - message passing between nodes

P
M
P
M
P
M
P
M
A
P
M
A
P
M
A